

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 1
по дисциплине: Вычислительная математика
по теме: Решение систем линейных алгебраических уравнений
(СЛАУ)

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Четвертухин В.Р.

Белгород 2024

Цель работы: Изучить методы решения СЛАУ и особенности их алгоритмизации в современных программных библиотеках NumPy, SciPy языка Python.

Первая часть данного задания предполагает решение СЛАУ вручную по классическому методу Гаусса.

Вторая часть задания предполагает написание и выполнение коротких программ на языке Python для решения той же СЛАУ (своего индивидуального задания) с использованием разных алгоритмических техник в интерактивном блокноте Jupyter.

Необходимо сравнить вычислительные схемы и полученные результаты для разных алгоритмов между собой и с результатами собственных вычислений вручную.

Третья часть задания — творческая. Нужно изменить параметры своего индивидуального задания так, чтобы представленные программы не справлялись с решением (полученное решение являлось бы неточным, либо программа не смогла бы получить решение и завершилась бы с ошибкой). Изучить типы полученных ошибок и проинтерпретировать численные ситуации.

Часть 1

Лабораторная работа №1 Вариант 7

$$\begin{cases} 185x_1 - 174x_2 + 208x_3 = -200 \\ 225x_1 - 193x_2 + 237x_3 = 220 \\ -217x_1 + 190x_2 - 239x_3 = -230 \end{cases}$$

Составлю матрицу для системы

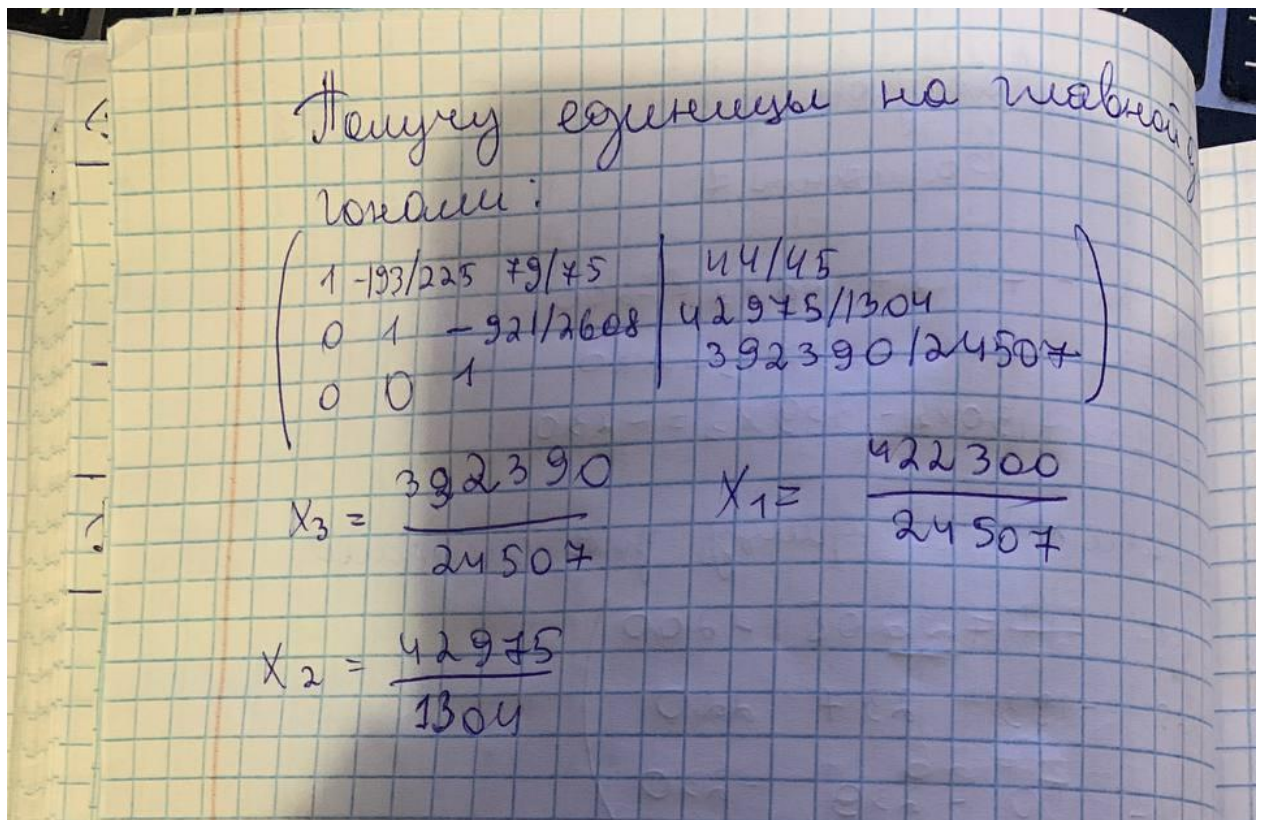
$$\left(\begin{array}{ccc|c} 185 & -174 & 208 & -200 \\ 225 & -193 & 237 & 220 \\ -217 & 190 & -239 & -230 \end{array} \right)$$

Добавлю II строку на $(k = 185/217)$ и добавлю к III:

$$\left(\begin{array}{ccc|c} 225 & -193 & 237 & 220 \\ 0 & 869/217 & -782/175 & -802/45 \\ 0 & -2608/217 & 921/217 & -85950/217 \end{array} \right)$$

Добавлю II строку на $(188573/586800)$ и добавлю к III

$$\left(\begin{array}{ccc|c} 225 & -193 & 237 & 220 \\ 0 & -2608/217 & 921/217 & -85950/217 \\ 0 & 0 & -590891/65200 & -8514863/586800 \end{array} \right)$$



Часть 2

"Использование библиотеки NumPy Основным методом, используемым в `numpy.linalg.solve` является метод LU-разложения (LU decomposition) и его вариации "

```
import numpy as np
# Коэффициенты системы уравнений
A = np.array([[ 185.0, -174.0, 288.0],
[ 225.0,-193.0, 237.0],
[-217.0, 190.0, -239.0]], dtype=float)
b = np.array([-200.0, 220.0, -230.0], dtype=float)
# Решение системы
x = np.linalg.solve(A, b)
print("Решение системы с использованием numpy.linalg.solve:", x)
```

Решение системы с использованием `numpy.linalg.solve`: [17.23181132 38.61060105 16.0113437]

"Использование библиотеки SciPy SciPy предлагает `scipy.linalg.lu_solve` для решения системы с помощью LU-разложения "

```
[10] from scipy.linalg import lu_factor, lu_solve
lu, piv = lu_factor(A)
x_lu = lu_solve((lu, piv), b)
print("Решение системы с использованием scipy.linalg:", x_lu)
```

Решение системы с использованием `scipy.linalg`: [17.23181132 38.61060105 16.0113437]

" Решение СЛАУ методом Гаусса. Метод Гаусса – классический алгоритм решения СЛАУ, основанный на преобразовании исходной системы к верхнетреугольной форме с последующим обратным ходом для нахождения решений. Параметры:

- A (numpy.ndarray): Матрица коэффициентов СЛАУ. Должна быть квадратной (n x n) и невырожденной.
- b (numpy.ndarray): Вектор свободных членов СЛАУ. Должен быть длины n, где n - количество уравнений в системе. Возвращает:
- numpy.ndarray: Вектор решения x системы $Ax = b$. Имеет размерность n, соответствующую числу уравнений в системе. "

```
[3] import numpy as np
# Коэффициенты системы уравнений
A = np.array([[ 185.0, -174.0, 208.0],
[ 225.0,-193.0, 237.0],
[-217.0, 190.0, -239.0]], dtype=float)
b = np.array([-200.0, 220.0, -230.0], dtype=float)
def gauss(A, b):
    numEquations = len(b)
    numEquations = len(b)
    # Прямой ход
    for pivotRow in range(numEquations):
        for currentRow in range(pivotRow + 1, numEquations):
            factor = A[currentRow, pivotRow] / \
                A[pivotRow, pivotRow]
            for currentCol in range(pivotRow, numEquations):
                A[currentRow, currentCol] -= factor * \
                    A[pivotRow, currentCol]
                b[currentRow] -= factor * b[pivotRow]

    # Обратный ход
    solutionVector = np.zeros(numEquations)
    for currentRow in range(numEquations - 1, -1, -1):
        sum_ax = 0
        for currentCol in range(currentRow + 1, numEquations):
            sum_ax += A[currentRow, currentCol] * \
                solutionVector[currentCol]
        solutionVector[currentRow] = (b[currentRow] - sum_ax) / \
            A[currentRow, currentRow]
    return solutionVector
print(gauss(A.copy(), b.copy()))
```

[17.23181132 38.61060105 16.0113437]

" Решает СЛАУ методом Гаусса с частичным выбором ведущего элемента. Этот метод улучшает точность вычислений за счет минимизации ошибок округления, выбирая в качестве ведущего элемента максимальный по модулю в текущем столбце.

Параметры:

- matrix (numpy.ndarray): квадратная матрица коэффициентов СЛАУ.
- vector (numpy.ndarray): вектор свободных членов СЛАУ. Возвращает:
- numpy.ndarray: вектор решения СЛАУ. "

```
def gauss_elimination_with_partial_pivoting(matrix, vector):
    matrix_size = len(matrix)
    # Прямой ход
    for current_column in range(matrix_size):
        # Поиск максимального элемента в текущем столбце
        max_index = np.argmax(np.abs(matrix[current_column:, current_column])) + \
            current_column
        # Обмен строк в матрице и векторе свободных членов
        matrix[[current_column, max_index]], \
            vector[[current_column, max_index]] = \
            (
                matrix[[max_index, current_column]],
                vector[[max_index, current_column]],
            )
        for i in range(current_column + 1, matrix_size):
            factor = matrix[i][current_column] / \
                matrix[current_column][current_column]
            matrix[i, current_column:] -= factor * \
                matrix[current_column, current_column:]
            vector[i] -= factor * vector[current_column]

    # Обратный ход
    solution = np.zeros(matrix_size)
    for i in range(matrix_size - 1, -1, -1):
        solution[i] = (vector[i] - np.dot(matrix[i, i + 1:], \
            solution[i + 1:])) / matrix[i][i]
    return solution
print(gauss_elimination_with_partial_pivoting(A, b))
```

[17.23181132 38.61060105 16.0113437]

" Решает СЛАУ методом Гаусса с частичным выбором ведущего элемента. Этот метод улучшает точность вычислений за счет минимизации ошибок округления, выбирая в качестве ведущего элемента максимальный по модулю в текущем столбце.

Параметры:

- matrix (numpy.ndarray): квадратная матрица коэффициентов СЛАУ.
- vector (numpy.ndarray): вектор свободных членов СЛАУ. Возвращает:
- numpy.ndarray: вектор решения СЛАУ. "

```
def gauss_elimination_with_partial_pivoting(matrix, vector):
    matrix_size = len(matrix)
    # Прямой ход
    for current_column in range(matrix_size):
        # Поиск максимального элемента в текущем столбце

        max_index = np.argmax(np.abs(matrix[current_column:, current_column])) + \
            current_column
        # Обмен строк в матрице и векторе свободных членов
        matrix[[current_column, max_index]], \
        vector[[current_column, max_index]] = \
        (
            matrix[[max_index, current_column]],
            vector[[max_index, current_column]],
        )
        for i in range(current_column + 1, matrix_size):
            factor = matrix[i][current_column] / \
                matrix[current_column][current_column]
            matrix[i, current_column:] -= factor * \
                matrix[current_column, current_column:]
            vector[i] -= factor * vector[current_column]
    # Обратный ход
    solution = np.zeros(matrix_size)
    for i in range(matrix_size - 1, -1, -1):
        solution[i] = (vector[i] - np.dot(matrix[i, i + 1:], \
            solution[i + 1:])) / matrix[i][i]
    return solution
print(gauss_elimination_with_partial_pivoting(A, b))
```

[3.76 1.44 1.04]

" Выполняет LU-разложение матрицы и решает СЛАУ с помощью этого разложения. LU-разложение преобразует матрицу A в произведение двух матриц: L (нижнетреугольная) и U (верхнетреугольная), что облегчает решение СЛАУ. Параметры:

- matrix (numpy.ndarray): квадратная матрица коэффициентов СЛАУ.
- vector (numpy.ndarray): вектор свободных членов СЛАУ. Возвращает:
- numpy.ndarray: вектор решения СЛАУ. "

```
def lu_decomposition(matrix, vector):
    matrix_size = len(matrix)
    L = np.zeros((matrix_size, matrix_size))
    U = np.zeros((matrix_size, matrix_size))
    # LU разложение
    for row in range(matrix_size):
        L[row, row] = 1
        for col in range(row, matrix_size):
            sum_upper = sum(L[row, sum_index] * U[sum_index, col] \
                for sum_index in range(row))
            U[row, col] = matrix[row, col] - sum_upper
        for col in range(row + 1, matrix_size):
            sum_lower = sum(L[col, sum_index] * U[sum_index, row] \
                for sum_index in range(row))
            L[col, row] = (matrix[col, row] - sum_lower) / U[row, row]

    # Решение Ly = b для y
    y = np.zeros(matrix_size)
    for row in range(matrix_size):
        y[row] = vector[row] - np.dot(L[row, :row], y[:row])
    # Решение Ux = y для x
    x = np.zeros(matrix_size)
    for row in range(matrix_size - 1, -1, -1):
        x[row] = (y[row] - np.dot(U[row, row + 1:], x[row + 1:])) / \
            U[row, row]
    return x
print(lu_decomposition(A, b))
```

[17.23181132 38.61060105 16.0113437]

Ссылка на блокнот:

<https://colab.research.google.com/drive/1mj0R-9EkBu3TsSRYer2fKFHLmJNcMOYh?usp=sharing>

Часть 3

Данные, при которых программа выдаёт ошибку

```
A = np.array([[-2, 3, 4, 5, 7, 10 ],
[ 8, 1, -3, 4, 5, 6 ],
[1, -4, 8, 9, -3, -1 ],
[6, 4, 1, 9, 12, 16]], dtype=float)
# Вектор свободных членов
b = np.array([50, -11, 87, 39], dtype=float)
```

```
[nan nan nan nan]
<ipython-input-13-da05e09748ec>:30: RuntimeWarning: invalid value encountered in scalar divide
  solutionVector[currentRow] = (b[currentRow] - sum_ax) / \
```

Вывод: Мы изучили методы решения СЛАУ и особенности их алгоритмизации в современных программных библиотеках NumPy, SciPy языка Python.